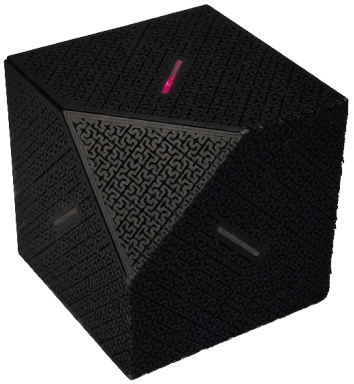
FLIPBUDDY

Assembly Guide

Firmware 0.2.0 · Super Mini DIY cube · flipbuddy.app

A compact, battery-powered, Wi-Fi cube you flip to track activities , a chess timer for daily tasks. Pair it with the free dashboard at flipbuddy.app, or use it fully offline.



Why this exists

FlipBuddy started as my constant search for a way to track time spent on each activity per day , so I could better understand the gap between actual time and my own perception of it. There are plenty of software tools and paid gadgets. I wanted a few things I could not find elsewhere, so I built FlipBuddy and released it as open source.

What I wanted

1. Must be open source
2. Cheap to source and easy to DIY
3. Simple to assemble as a hobby weekend project
4. No privacy compromise

The electronics are an off-the-shelf ESP32-S3 Super Mini plus an MPU6050 accelerometer , cheap and easy to find. The 3D-printed shell is fully customizable (STL and FreeCAD files in the GitHub repo).

You can use FlipBuddy 100% offline (dump JSON of recorded activities and write your own scripts). There is also a completely free companion app at https://flipbuddy.app that makes setup, colors, and history much easier.

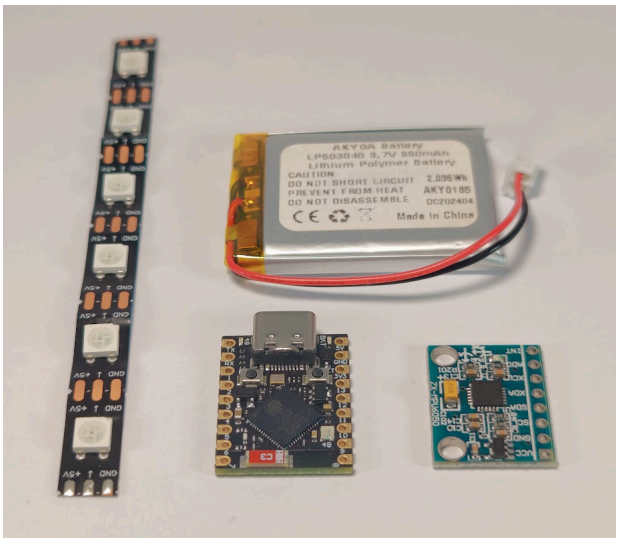
I am not an electronics engineer, product designer, or professional software developer , just a curious tinkerer. Treat this as a hobby project. You build at your own risk.

How a build goes (read this once)

Do the work in this order. Firmware is easier before the cube is snapped shut.

1. Print the tolerance check, pick a net, print clips and light guides.
2. Wire **Super Mini + MPU6050 + six LEDs + battery** (breadboard is fine for a first test).
3. Flash the FlipBuddy 0.2.0 OTA image, then put main.py and a SoftAP password on the cube.
4. Pair from the cube's own Wi-Fi with an Add Device code (not a credentials.json download).
5. Assemble the shell, clips, and guides. Power up and flip a face.

GitHub README is the source of truth for flash commands if this page and the repo ever disagree.



What you are building

A printed cube. Face up means that activity is on (not a countdown alarm). Flip when you switch. Put the stop face up when you are done. LEDs show the colors you set in the free app.

- Up to 8 faces (one can be stop)
- Deep sleep so a ~550 mAh pack lasts a while
- Pairing and later bugfix/security updates can happen without opening the cube (if you flashed the OTA image)

This is for minutes and hours, not a stopwatch.

Skills and time

Expect intermediate DIY: 3D print, solder, flash a board over USB. A careful first build often takes more than one evening after parts arrive. The tolerance-check print is worth the extra plastic.

LiPo safety: do not short the pack, use a proper charger (the Super Mini's USB-C charger is the usual path), and do not leave charging unattended.

Bill of materials

Buy parts that fit the shell cutout and show GPIO labels for pins 7–12. Shop links go stale; the Amazon examples below are only starting points.

Qty	Part	Notes
1	ESP32-S3 Super Mini, USB-C, onboard charger preferred	4 MB flash. This is the board the frozen image is built for.
1	MPU6050 / GY-521	Face detection. Mount it as in the photos or faces will be wrong.
1	3.7 V protected LiPo ~550 mAh	Smaller works; runtime shrinks.
6	WS2812B 5050 RGB LEDs	One per face. 5 V data from the Super Mini.
2	470 kΩ resistors (optional)	Battery ADC divider if you want a voltage reading.

Example listings (check fit before you order):

Super Mini: amazon.de/dp/B0D8Z93QDS

MPU6050: amazon.de/dp/B0DTLS3HN5

WS2812: amazon.de/dp/B01D1GL2N4

550 mAh LiPo: amazon.de/dp/B0BRT9Y211

Notes on alternatives

Other ESP32 modules can work if you change the FreeCAD cutout and pin map (main.py / mpu6050.py). You then flash stock MicroPython and copy every .py file (just diy). That path has no self-OTA.

Development of a custom “system on module” with BQ21061 charger, TPS63900 boost, BQ27427 fuel gauge, and Bosch BMA400 accelerometer in a compact 25×25 mm footprint is underway. Early samples measured ~8.5 µA in deep sleep. Stay tuned.

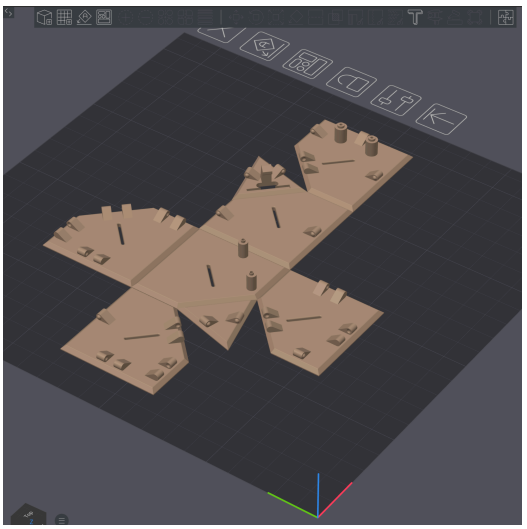
Parts to print

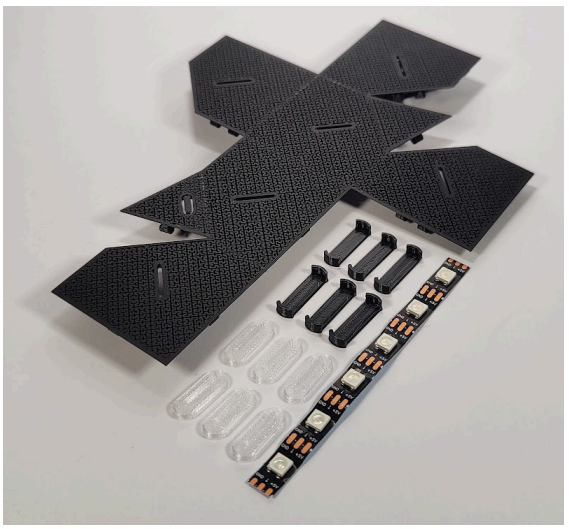
All STLs are in the repo stl/ folder. Filenames include a small design revision tag.

File	Role	Qty
tolerance_check.stl	Print first. Pick a net from the result.	1
tolerance_20_…_net.stl … tolerance_40_…_net.stl	Shell net (tighter → looser)	1
flipbuddy_con_v0.1.0_led_clip.stl	Holds one WS2812	6
flipbuddy_con_v0.1.0_led_light_guide.stl	Light pipe into the face	6
flipbuddy_con_v0.1.0_new_mpu6050_clip.stl	Holds the sensor	1
flipbuddy_con_v0.1.0_new_led.FCStd	FreeCAD (edit cutouts / pins)	,

Print tips

- **PLA** or **PETG**, ~0.2 mm layers, works for many printers.
- Print light guides slower in **PETG** and calibrate flow , they are small and delicate.
- Take your time with the first tolerance sample. It saves reprints of the whole net.
- USB-C cutout and board mounts can be tweaked in FreeCAD for a different module.





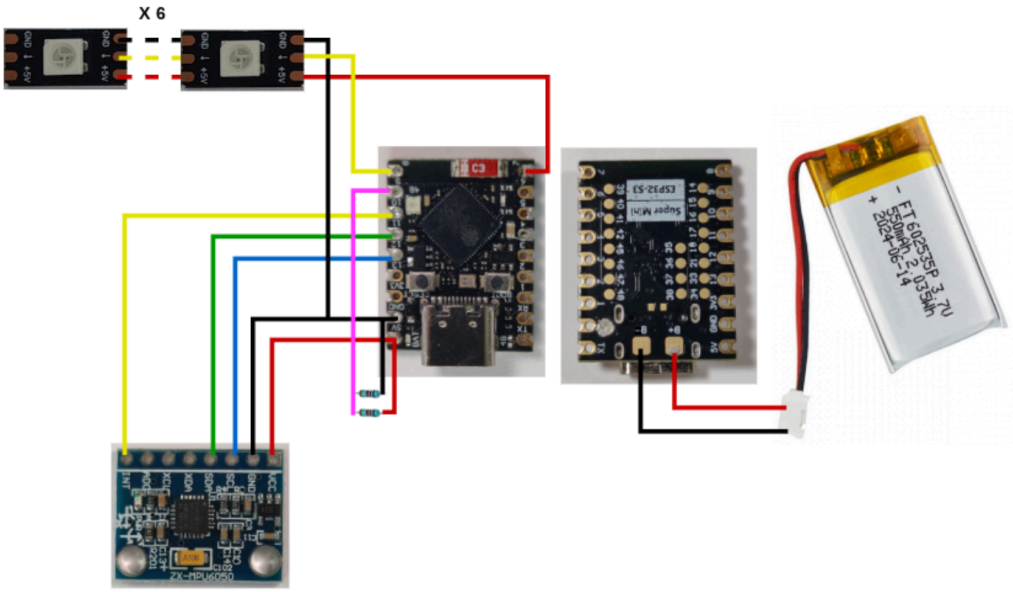
Wiring (Super Mini)

Use the GPIO numbers on the silkscreen. Super Mini clones are not all identical.

Function	GPIO
NeoPixel VCC gate	7
NeoPixel data	8
Battery ADC	9
MPU INT	10
I2C SDA	11
I2C SCL	12
Onboard LED (if used)	48

LED index order in firmware (DIN starts at 0): front, bottom, back, left, top, right.

Worth a breadboard test before final soldering: flash, pair, one LED on 7/8, MPU on 10–12, serial at 115200, shake to wake.



Flash and pair (firmware 0.2.0)

Do this before you glue and snap the shell if you can. USB-C needs to stay reachable.

This is a claim pairing. You do not start with a credentials.json from the dashboard. That file is recovery only.

v0.1.0–v0.1.5 cubes: those images do not work with current pairing or over-the-air updates. USB-flash 0.2.0-ota.bin below, then pair again. You cannot OTA from 0.1.5 onto 0.2.0.

What to download

From this project's GitHub Releases → v0.2.0:

File	Use
esp32_s3_flipbuddy_0.2.0-ota.bin	Flash this on Super Mini. Dual-OTA table. Later updates can come from flipbuddy.app.
esp32_s3_flipbuddy_0.2.0.bin	USB recovery only (no self-OTA).
*.bin.sha256 / SHA256SUMS	Checksums

Do not flash stock MicroPython from micropython.org on Super Mini if you can avoid it. The FlipBuddy image already contains pairing helpers and can pull tested bugfix/security binaries. Stock is for other chips, or if you want every .py on the filesystem (just diy, no self-OTA).

1. Verify and flash (Linux example)

Port is often /dev/ttyACM0 (also try /dev/ttyUSB0, COM3, /dev/cu.usbmodem*). Hold BOOT, tap RST, release BOOT if the board will not enter download mode.

```
sha256sum -c esp32_s3_flipbuddy_0.2.0-ota.bin.sha256
esptool --chip esp32s3 --port /dev/ttyACM0 erase-flash
esptool --chip esp32s3 --port /dev/ttyACM0 --baud 460800 write-flash 0
esp32_s3_flipbuddy_0.2.0-ota.bin
```

Same with just (looks in the current directory, then build/):

```
just flash-firmware esp32_s3_flipbuddy_0.2.0-ota.bin
```

Erase-flash wipes the pairing token, house Wi-Fi, and SoftAP password. After every full flash you pair again. Skip "Forget FlipBuddy account" , the cube is already empty. Use Add Device.

2. Put main.py and a join password on the cube

Helpers are already inside the frozen image. You only add the entrypoint and a random SoftAP password:

```
just fast-track
```

That uploads main.py and ap_psk.json, then resets. It does not copy credentials.json, even if that file is sitting next to the justfile.

Write down the password. Serial / the host print it as XXXX XXXX (spaces are only for reading). Type it without the space.

If serial says FATAL: no SoftAP join password:

```
just gen-ap-psk
just put-ap-psk
```

Then power-cycle. The image will not invent a password.

Without just:

```
python3 scripts/gen_ap_psk.py
mpremote connect /dev/ttyACM0 cp main.py :
mpremote connect /dev/ttyACM0 cp ap_psk.json :
mpremote connect /dev/ttyACM0 reset
```

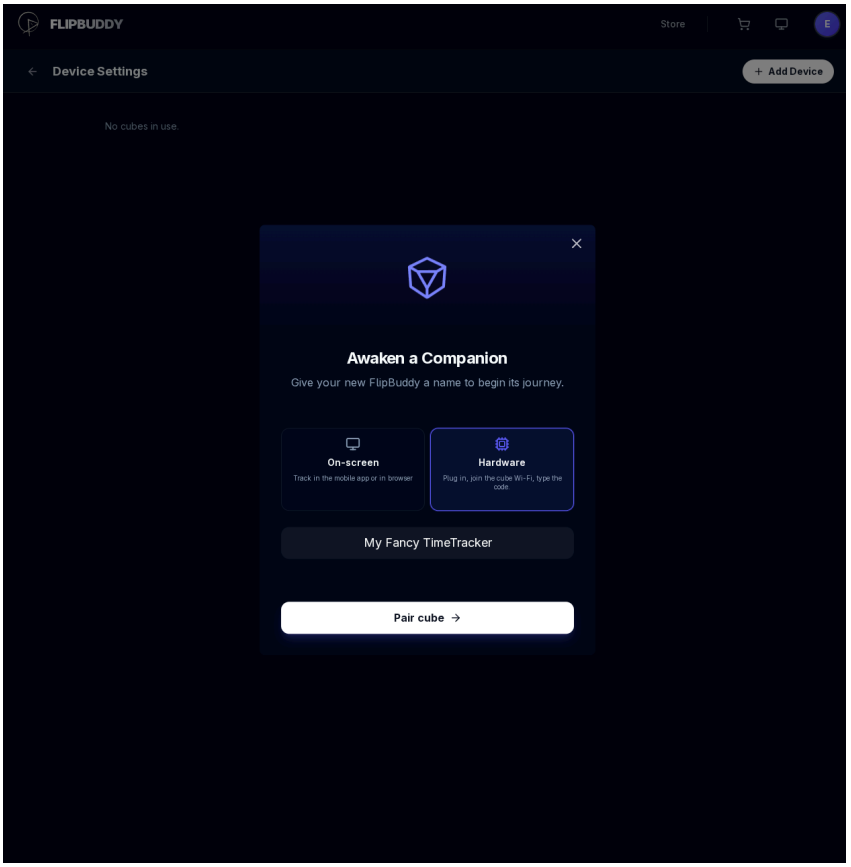
3. Claim from the cube Wi-Fi

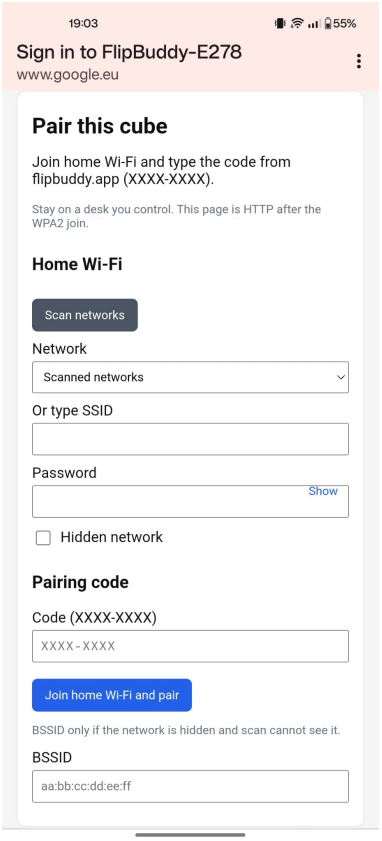
Keep USB plugged in until the cube blinks green and the app confirms.

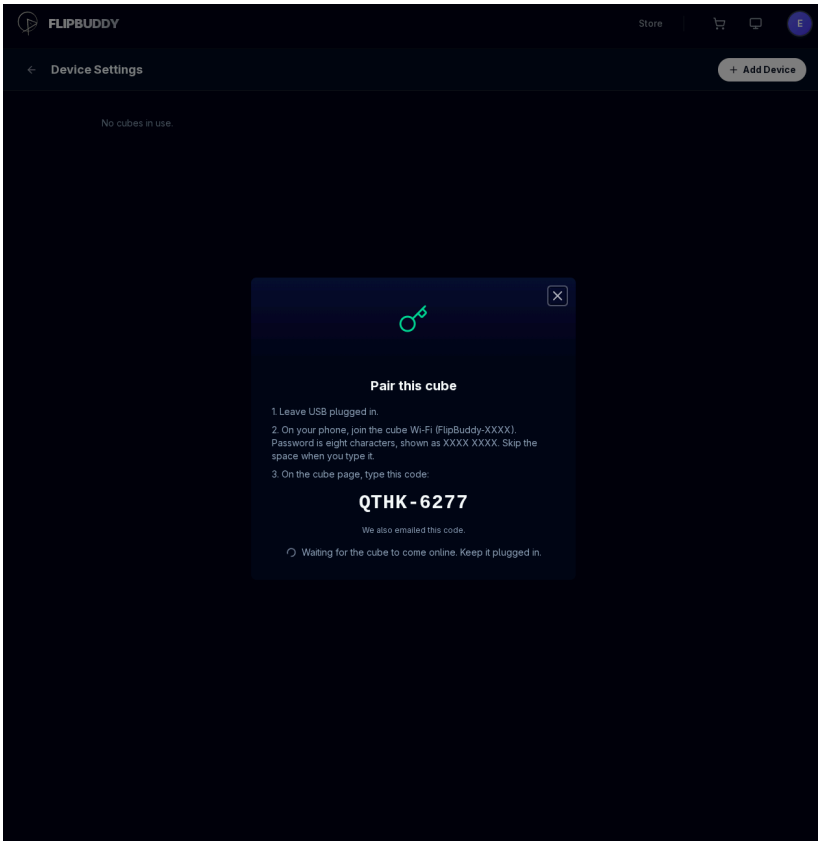
1. Create a free account at flipbuddy.app.
2. Add Device → physical cube → give it a name. The wizard shows an 8-character code (XXXX-XXXX).
3. On your phone, join Wi-Fi **FlipBuddy-XXXX** (last 4 of the chip id, for example FlipBuddy-323C). Password = the just fast-track password, no space.
4. Captive portal, or open http://10.20.30.40/. Enter house Wi-Fi plus the dashboard pairing code. House SSID/password stay on the cube. They are never sent to the API.
5. Cube blinks **green**. In the app, confirm / Start tracking.

Unclaimed serial looks like:

```
Unclaimed cube.
  Pair from the cube Wi-Fi. Keep USB plugged in.
```







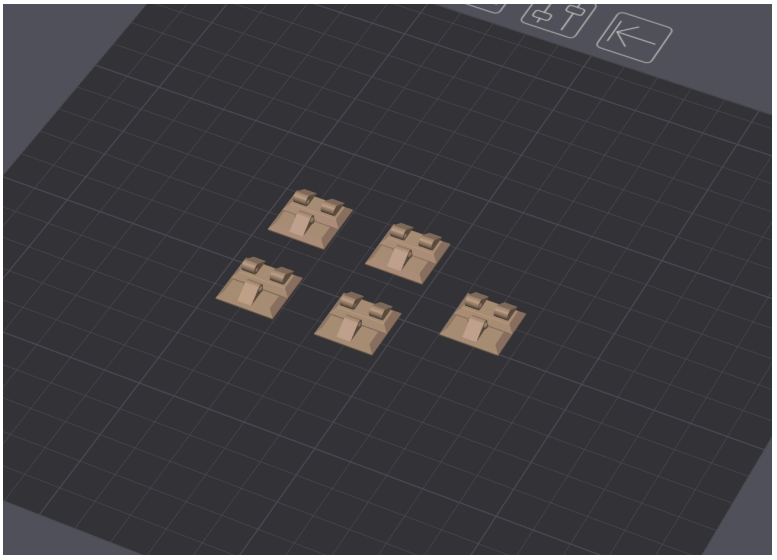
Recovery later (lost token, 60-day expiry, and so on): download JSON from the device page and run just put-credentials. That restores identity only. It is not the first-time path.

Other boards / stock MicroPython: flash a GENERIC image for your chip from micropython.org (1.28 matches the Release), then just diy (all modules + SoftAP password). Pairing is the same claim flow. You apply updates with USB; the cube will not self-flash.

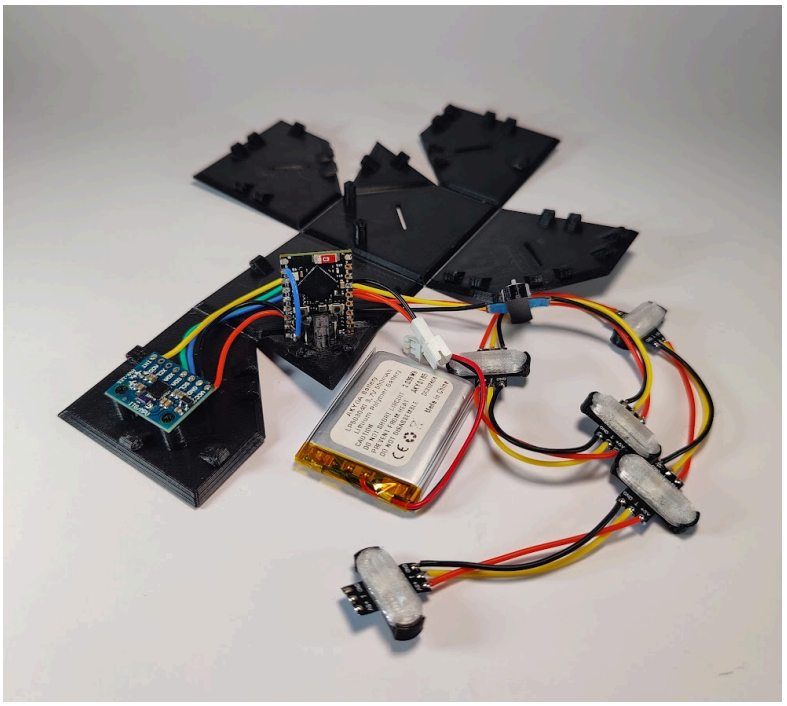
Mechanical assembly

Flash and a quick LED/MPU test first if you can.

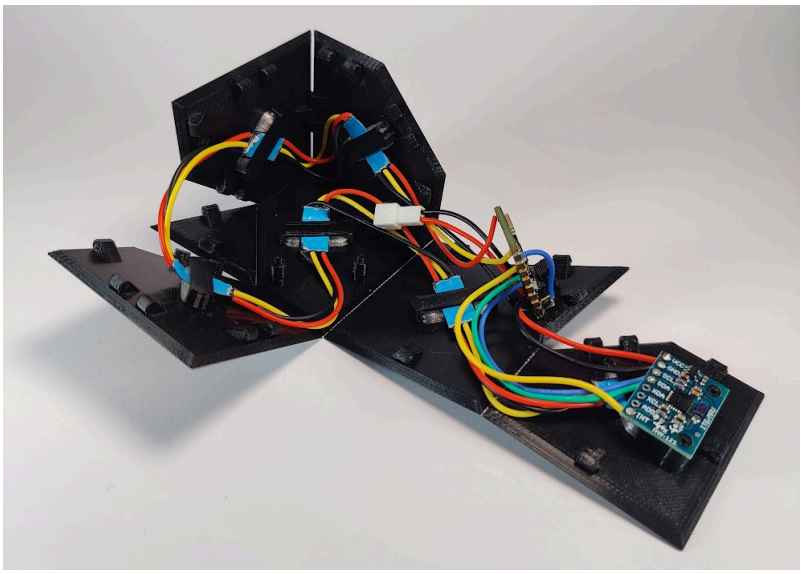
1. Print the tolerance check, pick the net that fits, print that net plus clips and guides.



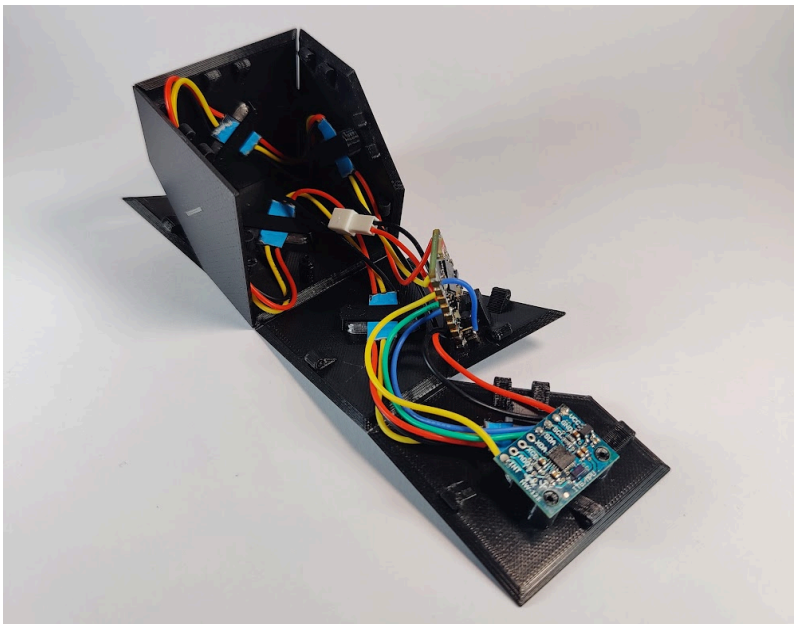
2. LED clips , mount each WS2812 in a clip. The clips rotate around their center to lock. Do not only push them straight down.



3. Insert the light guides into the faces.



4. Seat the electronics , Super Mini in the USB-C cutout, MPU in its clip (orientation as in the photo), battery, LED chain. Keep the USB-C port aligned with the shell cutout (that face is also SoftAP later).



5. Fold the net (or assemble individual faces) and snap together. Glue only where the guide says; many joints are snap-fit.



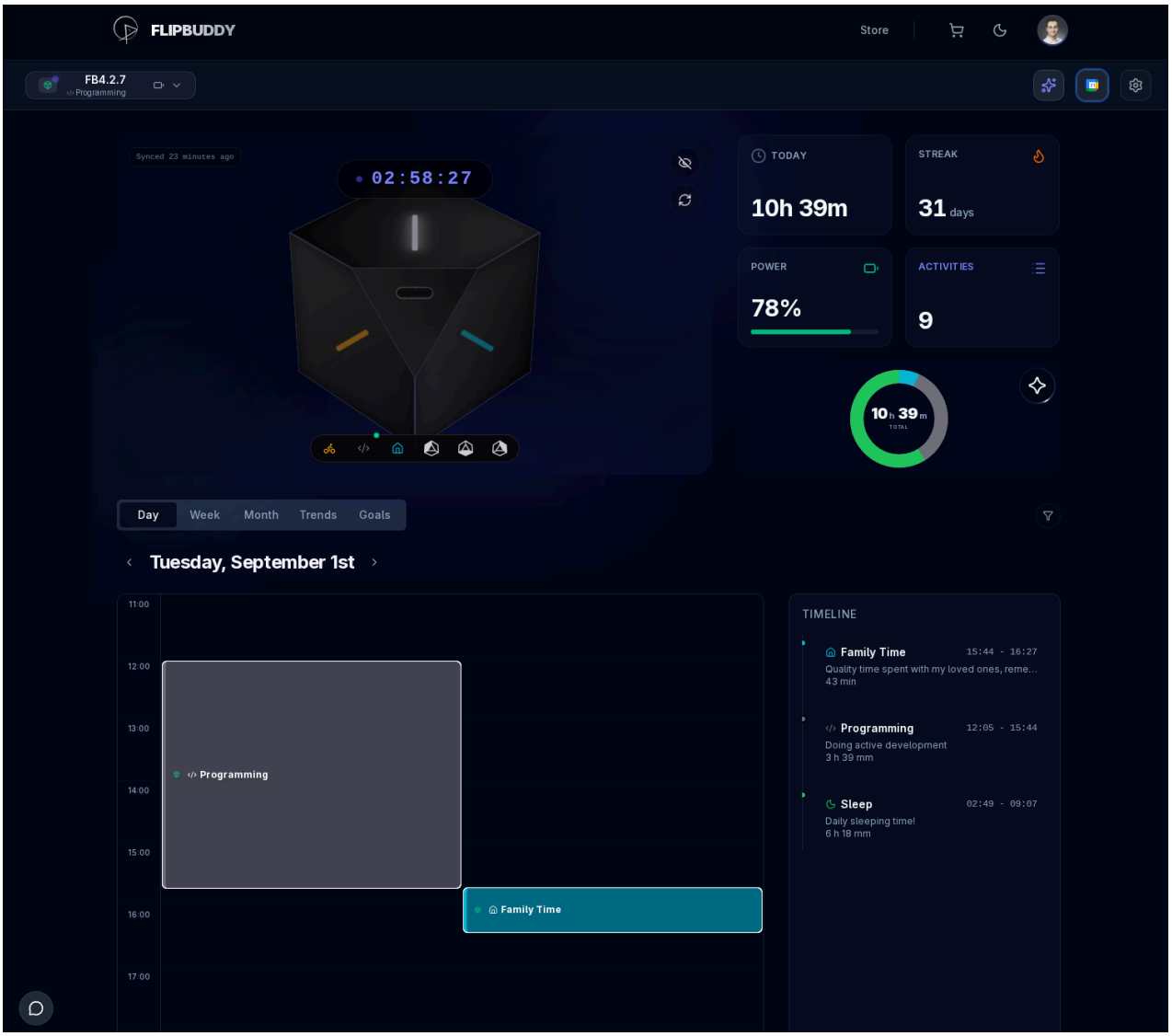
6. Done. Power it, flip a face, watch the LEDs.

After it is together

On first boot after a successful claim the cube tries Wi-Fi, calibrates, fetches face names/colors, then tracks flips. Shake if it went to sleep.

USB-C face up opens a short SoftAP session (about 5 minutes) so you can fix Wi-Fi without **mpremote**. After pairing, SSID is still **FlipBuddy-XXXX** if the cube has a host password; some claimed cubes without that password use SSID FlipBuddy. Portal: http://10.20.30.40/. SoftAP LEDs are solid yellow on the USB-face triad.

OTA (only if you flashed *-ota.bin): leave OTA on in device settings. When flipbuddy.app has a newer app-slot package, the cube pulls it over HTTPS. All six LEDs stay solid while it downloads. A bad file stays on the current slot.



If something looks wrong

What you see	What to try
All six LEDs cyan / light blue when you shake	No activities mapped yet. Set faces in the app, let it sync, shake again.
No FlipBuddy-XXXX Wi-Fi	Keep USB plugged in for pairing. Shake. USB-C face up. After a SoftAP session, flip off that face, then USB face up again.
FATAL: no SoftAP join password	just gen-ap-psk && just put-ap-psk, power-cycle.
Cube never joins house Wi-Fi	Type SSID/password on the pairing page (first time) or SoftAP later. Leave the FlipBuddy network and rejoin home Wi-Fi on the phone.
Faces are “working” but wrong	MPU orientation does not match the photo.
Times are wrong	Home Wi-Fi + NTP. SoftAP → Reset device, then an activity/stop face (not a cutout).
Token / dashboard after a long time off	Device token lasts up to 60 days. Fresh JSON + just put-credentials (recovery).
You USB-flashed and the app talks about Forget	Skip Forget. Add Device. Forget is only if the cube still thinks it is claimed.

Serial console: just shell or any 115200 port. Redact tokens and Wi-Fi passwords if you file a bug.

Tips

- Tweaking another ESP32 module: change the FreeCAD USB-C cutout and the pin table, then use the DIY/stock path.
- One cube at a time if you use two: stop one in the app before flipping the other a lot.
- Hardware license is CC BY-NC-SA 4.0 (shell, STLs, this guide). Firmware sources are MIT. Ask before commercial use of the enclosure.

If you build one, I would love to see photos or hear how it went. Feedback makes this better for everyone. Happy building.

Links

- App: https://flipbuddy.app
- Firmware + README (flash commands): GitHub Releases v0.2.0 and the repository README
- Hackaday: https://hackaday.com/2026/08/07/flipbuddy-is-borg-you-will-be-assimilated/